Version: 2.6



Fly-in

Drones are interesting.

Summary

Design an efficient drone routing system that navigates multiple drones through connected zones while minimizing simulation turns and handling movement constraints.

#Python #Group #Parsing



Intellectual Property Disclaimer

All content presented in this training module, including but not limited to texts, images, graphics, and other materials, is protected by intellectual property rights held by Association 42.

Terms of Use:

- **Personal use:** You are permitted to use the contents of this module solely for personal purpose. Any commercial use, reproduction, distribution, modification, or public display is strictly prohibited without prior written permission from Association 42.
- **Respect for Integrity:** You must not alter, transform, or adapt the content in any way that could harm its integrity.

Protection of Rights:

Any violation of these terms constitutes an infringement of intellectual property rights and may result in legal action. We reserve the right to take all necessary measures to protect our rights, including but not limited to claims for damages.

For any questions regarding the use of the content or to obtain authorization, please contact: `legal@42.fr`

Contents

1 Foreword **1**

2 AI Instructions **2**

3 Common Instructions **4**
3.1 General Rules 4
3.2 Makefile 4
3.3 Additional Guidelines 5

4 Introduction **6**

5 Constraints **7**

6 Let the drone fly **8**

7 Mandatory Part **11**
7.1 Pathfinding and Algorithm Requirements 11
7.2 Zone Occupancy Rules 12
7.3 Movement and Turn Mechanics 12
7.4 Parser Constraints 13
7.5 Simulation Output Format 15
7.6 Scoring System 15
7.7 Performance Benchmarks 16

8 Readme Requirements **19**

9 Submission and peer-evaluation **20**

Chapter 1

Foreword

Drones have been used to herd sheep in New Zealand, replacing sheepdogs with buzzing aerial shepherds. In Japan, office buildings deploy drones that play loud music and flash lights to literally chase overworked employees home. One drone was trained to paint graffiti on walls mid-flight — a rebellious blend of tech and street art. In Sweden, scientists used drones to sniff out whale poop floating on the ocean to study endangered species. Some experimental drones are shaped like birds or insects to spy without being noticed, flapping wings and all. There's even a drone that flies by flapping soap bubbles, no propellers involved. In volcano research, a drone once flew straight into an eruption cloud, melted mid-air, but managed to send back data just seconds before disintegration. And in South Korea, synchronized drone shows have replaced fireworks — safer, silent, and somehow even more magical.

The phrase comes from the idea that the wheel is a brilliant invention that's been around forever and works really well. Since there's nothing wrong with it, trying to invent it again wouldn't really help and could be a waste of time—especially when that time could be spent solving new problems instead.

In programming, this happens when someone builds something from scratch that already exists—like writing your own sorting algorithm or framework when solid, open-source versions are already out there. But it's not all bad: doing it yourself can be a great way to learn how things work under the hood. The key is to find a balance—don't rebuild everything, but take time to explore how the tools you're using actually work. That way, you'll grow as a developer without getting stuck reinventing the same old wheels.

© 2026 Association 42. - All rights reserved

Chapter 2

AI Instructions

Context

During your learning journey, AI can assist with many different tasks. Take the time to explore the various capabilities of AI tools and how they can support your work. However, always approach them with caution and critically assess the results. Whether it's code, documentation, ideas, or technical explanations, you can never be completely sure that your question was well-formed or that the generated content is accurate. Your peers are a valuable resource to help you avoid mistakes and blind spots.

Main message

- Use AI to reduce repetitive or tedious tasks.
- Develop prompting skills — both coding and non-coding — that will benefit your future career.
- Learn how AI systems work to better anticipate and avoid common risks, biases, and ethical issues.
- Continue building both technical and power skills by working with your peers.
- Only use AI-generated content that you fully understand and can take responsibility for.

Learner rules:

- You should take the time to explore AI tools and understand how they work, so you can use them ethically and reduce potential biases.
- You should reflect on your problem before prompting — this helps you write clearer, more detailed, and more relevant prompts using accurate vocabulary.
- You should develop the habit of systematically checking, reviewing, questioning, and testing anything generated by AI.
- You should always seek peer review — don't rely solely on your own validation.

© 2026 Association 42. - All rights reserved

Phase outcomes:

- Develop both general-purpose and domain-specific prompting skills.
- Boost your productivity with effective use of AI tools.
- Continue strengthening computational thinking, problem-solving, adaptability, and collaboration.

Comments and examples:

- You'll regularly encounter situations — exams, evaluations, and more — where you must demonstrate real understanding. Be prepared, keep building both your technical and interpersonal skills.
- Explaining your reasoning and debating with peers often reveals gaps in your understanding. Make peer learning a priority.
- AI tools often lack your specific context and tend to provide generic responses. Your peers, who share your environment, can offer more relevant and accurate insights.
- Where AI tends to generate the most likely answer, your peers can provide alternative perspectives and valuable nuance. Rely on them as a quality checkpoint.

✓ Good practice:

> I ask AI: "How do I test a sorting function?" It gives me a few ideas. I try them out and review the results with a peer. We refine the approach together.

✗ Bad practice:

> I ask AI to write a whole function, copy-paste it into my activity. During peer-evaluation, I can't explain what it does or why. I lose credibility — and I fail my activity.

✓ Good practice:

> I use AI to help design a parser. Then I walk through the logic with a peer. We catch two bugs and rewrite it together — better, cleaner, and fully understood.

✗ Bad practice:

> I let Copilot generate my code for a key part of my activity. It compiles, but I can't explain how it handles pipes. During the evaluation, I fail to justify and I fail my activity.

© 2026 Association 42. - All rights reserved

Chapter 3

Common Instructions

3.1 General Rules

- Your activity must be written in **Python 3.10 or later**.
- Your activity must adhere to the **flake8** coding standard.
- Your functions should handle exceptions gracefully to avoid crashes. Use `try-except` blocks to manage potential errors. Prefer context managers for resources like files or connections to ensure automatic cleanup. If your program crashes due to unhandled exceptions during the review, it will be considered non-functional.
- All resources (e.g., file handles, network connections) must be properly managed to prevent leaks. Use context managers where possible for automatic handling.
- Your code must include type hints for function parameters, return types, and variables where applicable (using the `typing` module). Use `mypy` for static type checking. All functions must pass mypy without errors.
- Include docstrings in functions and classes following PEP 257 (e.g., Google or NumPy style) to document purpose, parameters, and returns.

3.2 Makefile

Include a `Makefile` in your activity to automate common tasks. It must contain the following rules (mandatory lint implies the specified flags; it is strongly recommended to try `-strict` for enhanced checking):

- **install**: Install activity dependencies using `pip`, `uv`, `pipx`, or any other package manager of your choice.
- **run**: Execute the main script of your activity (e.g., via Python interpreter).
- **debug**: Run the main script in debug mode using Python's built-in debugger (e.g., pdb).
- **clean**: Remove temporary files or caches (e.g., `__pycache__`, `.mypy_cache`) to keep the activity environment clean.

© 2026 Association 42. - All rights reserved

- **lint**: `flake8 .` and `mypy . --warn-return-any --warn-unused-ignores --ignore-missing-imports --disallow-untyped-defs --check-untyped-defs`
  Run with: `make lint`.
- **lint-strict** (optional): `flake8 .` and `mypy . --strict`
  Run with: `make lint-strict`.

3.3 Additional Guidelines

- Create test programs to verify activity functionality (not submitted or graded). Use frameworks like `pytest` or `unittest` for unit tests, covering edge cases.
- Include a `.gitignore` file to exclude Python artifacts.
- It is recommended to use virtual environments (e.g., venv or conda) for dependency isolation during development.

If any additional activity-specific requirements apply, they will be stated immediately below this section.

© 2026 Association 42. - All rights reserved

Chapter 4

Introduction

Autonomous drones are the future of transportation. They are already used in many industries, such as agriculture, construction, and logistics. They are also used in military operations, such as surveillance and reconnaissance.

Your task is to design a system that **efficiently routes a fleet of drones** from a **central base (start)** to a target **location (end)**, while navigating this dynamic network under a set of strict constraints and optimization goals.

You'll be given a **graph** representing the network of zones, and a **set of constraints** that you must respect.

The graph is represented as a network of connected zones, where connections define possible movement paths between zones.

© 2026 Association 42. - All rights reserved

Chapter 5

Constraints

- Any library that helps with graph logic is forbidden. (such as networkx, graphlib, etc.)
- The project must be completely typesafe. Using **flake8** and **mypy** is mandatory.
- The project must be completely **object-oriented**.

This will have to be demonstrated during the peer review. Failing any of these constraints (forbidden library, **flake8**/**mypy** errors, or a non object-oriented design) is a blocking condition: see the evaluation scale's stop conditions.

© 2026 Association 42. - All rights reserved

Chapter 6

Let the drone fly

Attached to this subject, you'll find multiple files that represent the network of zones in the following format:

Example:

```
nb_drones: 5

start_hub: hub 0 0 [color=green]
end_hub: goal 10 10 [color=yellow]
hub: roof1 3 4 [zone=restricted color=red]
hub: roof2 6 2 [zone=normal color=blue]
hub: corridorA 4 3 [zone=priority color=green max_drones=2]
hub: tunnelB 7 4 [zone=normal color=red]
hub: obstacleX 5 5 [zone=blocked color=gray]
connection: hub-roof1
connection: hub-corridorA
connection: roof1-roof2
connection: roof2-goal
connection: corridorA-tunnelB [max_link_capacity=2]
connection: tunnelB-goal
```

© 2026 Association 42. - All rights reserved

Interesting, right? To be more precise:

- The first line defines the number of drones using `nb_drones: <number>`.
- Zone definition on each line using type prefixes:
  - `start_hub: <name> <x> <y> [metadata]` marks the starting zone.
  - `end_hub: <name> <x> <y> [metadata]` marks the end zone.
  - `hub: <name> <x> <y> [metadata]` defines a regular zone.
- The connection syntax forbids dashes in zone names (see below).
- All metadata is optional and enclosed in brackets `[...]` with default values:
  - `zone=<type>` (default: `normal`)
  - `color=<value>` (default: none)
  - `max_drones=<number>` (default: 1) - Maximum drones that can occupy this zone simultaneously
  - Tags inside brackets can appear in any order.
- Zone types:
  - `normal` – Standard zone with 1 turn movement cost (default)
  - `blocked` – Inaccessible zone. Drones must not enter or pass through this zone. Any path using it is invalid.
  - `restricted` – A sensitive or dangerous zone. Movement to this zone costs 2 turns.
  - `priority` – A preferred zone. Movement to this zone costs 1 turn but should be prioritized in pathfinding.
- Colors:
  - Colors are optional and can be used for visual representation (terminal output or graphical display).
  - Accepted values for `color` are any valid single-word strings (e.g., `red`, `blue`, `gray`). There is no fixed list of allowed colors.
  - When colors are specified, the implementation should provide visual feedback through colored terminal output or graphical representation.
- Connections are defined using `connection: <name1>-<name2> [metadata]`:
  - Defines a bidirectional connection (edge) between two zones.
  - The connection syntax forbids dashes in zone names.
  - Optional metadata can be specified in brackets `[...]`:
    - `max_link_capacity=<number>` (default: 1) - Maximum drones that can traverse this connection simultaneously
- Comments start with '#' and are ignored.

© 2026 Association 42. - All rights reserved



The zones coordinates will always be integers, and there will always be a unique start and a unique end zone.

© 2026 Association 42. - All rights reserved

Chapter 7

Mandatory Part

As you may have guessed, the main objective is to move all drones from the start zone to the end zone in the fewest possible simulation turns.

7.1 Pathfinding and Algorithm Requirements

- Drones may move simultaneously. The algorithm must schedule paths to maximize throughput and avoid unnecessary delays.
- Your implementation must handle:
  - Distribution of drones across multiple paths.
  - Strategic waiting when movement is not possible.
  - Avoidance of path conflicts and deadlocks.
- The algorithm must take into account:
  - Path lengths, including movement costs associated with zone types (e.g., restricted or priority).
  - Turn scheduling, to prevent drones from colliding or blocking each other.
  - Graph structure, to determine available disjoint or overlapping paths.
  - Zone capacity constraints (max_drones) and connection capacity (max_link_capacity).
- Your algorithm should be adaptable: different maps may require different routing strategies, depending on the topology and zone types.
- **Visual Representation**: Your implementation must provide visual feedback of the simulation, either through:
  - Colored terminal output showing drone movements and zone states
  - A graphical interface displaying the network and drone positions
  - Both options for enhanced user experience

© 2026 Association 42. - All rights reserved



How efficient is your algorithm?
Can it work with a large number of drones?
What is the complexity (e.g., O(n), O(log n), etc.)?
Are you recalculating or caching paths?
How does it impact memory usage?
How does your visual representation enhance understanding of the simulation?

7.2 Zone Occupancy Rules

- By default, a zone may contain at most one drone at any given simulation turn.
- Zones with `max_drones=N` metadata can contain up to N drones simultaneously.
- The only special exceptions to occupancy rules are:
  - The `start` zone: all drones begin here and may share the space initially.
  - The `end` zone: multiple drones can arrive here and are considered delivered.
- Two drones may not enter the same zone on the same turn unless the zone's capacity allows it.
- A drone may not move into a zone that would exceed its maximum capacity.
- Connection capacity (`max_link_capacity`) defined on connections limits how many drones can traverse the same connection simultaneously.
- Drones may move simultaneously, as long as all capacity constraints are respected.

7.3 Movement and Turn Mechanics

The simulation proceeds in discrete turns. At each turn, every drone may:

- Move to an adjacent connected zone (if capacity allows).
- Move to a connection towards a restricted zone (that requires 2 turns to be reached). In this case, the drone MUST reach its destination during the next turn. It can't wait extra turns on the connection.
- Stay in place (e.g., to wait, or if movement is blocked).

The simulation must prevent conflicts and ensure valid movement scheduling based on turn-by-turn state evaluation:

- All drone movements within a turn are simultaneous: every drone's move is decided from the state at the start of the turn, anticipating what is possible or not. There is no change of mind in the middle of the turn.
- Drones moving out of a zone or off a connection free up that capacity for that same turn: a space vacated this turn is available to another drone during that same turn (e.g. a single-file column of drones can advance one step per turn).

© 2026 Association 42. - All rights reserved

- A zone must have available capacity for a drone to move into it (after all drones moving out have freed up space).
- For multi-turn movements (restricted zones), the drone occupies the connection during transit and MUST arrive at the destination after the specified number of turns. It cannot wait on the connection for an empty space in the destination zone.
- A restricted connection is freed as soon as the drone departs from it, i.e. on the turn the drone arrives in the destination zone. Another drone may start crossing that same connection during that same turn, provided every other movement and capacity constraint is respected.

Example: two drones queueing for a restricted connection: A connection with `max_link_capacity=1` leads to a restricted zone. Drone D1 starts crossing it at turn 1 and arrives in the restricted zone at turn 2, which frees the connection from turn 2 onward. Drone D2, waiting at the connection's origin zone, may start its own crossing at turn 2 and will arrive at turn 3:

```
D1-restrictedConn
D1-restrictedZone D2-restrictedConn
D2-restrictedZone
```

Each movement between zones has a cost in turns, based on the **zone=type** of the destination:

- `normal`: 1 turn (default)
- `restricted`: 2 turns
- `priority`: 1 turn (but should be preferred in pathfinding algorithms)
- `blocked`: Inaccessible — cannot be entered

7.4 Parser Constraints

The input file must respect the expected structure and syntax:

- The first line must define the number of drones using `nb_drones: <positive_integer>`.
- The program must be able to handle any number of drones.
- There must be exactly one `start_hub:` zone and one `end_hub:` zone.
- Each zone must have a unique name and valid integer coordinates.
- Zone names can use any valid characters except dashes and spaces.
- Connections must link only previously defined zones using `connection: <zone1>-<zone2> [metadata]`.
- The same connection must not appear more than once (e.g., `a-b` and `b-a` are considered duplicates).
- Any metadata block (e.g., `[zone=... color=...]` for zones, `[max_link_capacity=...]` for connections) must be syntactically valid.

© 2026 Association 42. - All rights reserved

- Zone types must be one of: `normal`, `blocked`, `restricted`, `priority`. Any invalid type must raise a parsing error.
- Capacity values (`max_drones` for zones, `max_link_capacity` for connections) must be positive integers.
- The `max_drones` capacity is ignored on the `start_hub` and `end_hub` zones: these have no capacity limit (all drones may start in the start zone, and any number of drones may be delivered to the end zone). If such metadata is present on those two zones, it is ignored and is not a validation error.
- Any other parsing error must stop the program and return a clear error message indicating the line and cause.

A syntactically valid map can still be unsolvable, in particular when no path connects the `start_hub` to the `end_hub` (a disconnected graph). This is not a parsing error: the program must detect it and report it clearly instead of crashing or running indefinitely.



It's highly recommended to make your own map files on top of the ones provided in the subject for handling edge cases and error handling.

© 2026 Association 42. - All rights reserved

7.5 Simulation Output Format

- The simulation must output the step-by-step movement of drones from the start to the end zone.
- Each simulation turn is represented by a line.
- A line must list all the drone movements that occur during that turn, space-separated. Each movement must follow the format: `D<ID>-<zone>`, or `D<ID>-<connection>` in case of drones still in flight toward restricted zones.
  - `D<ID>` refers to the unique drone identifier (e.g., `D1`, `D2`).
  - `<zone>` is the name of the destination zone.
  - `<connection>` is the name of the connection toward a restricted zone.
- Drones that do not move in a given turn are omitted from that line.
- Drones that reach the end zone are considered delivered and are no longer tracked.
- The simulation ends when all drones have reached the end zone.
- Example:

```
D1-roof1 D2-corridorA
D1-roof2 D2-tunnelB
D1-goal D2-goal
```

7.6 Scoring System

- A valid simulation must:
  - Comply with all movement and occupancy rules.
  - Correctly handle movement costs associated with zone types.
  - Respect all capacity constraints (zone and connection limits).
  - Avoid all conflicts (e.g., exceeding zone or connection capacity).
- The performance of a solution is evaluated based on the **total number of simulation turns** required to route all drones from the start zone to the end zone. Your solution will be compared to the optimum for each map as described below.



Speed never causes a submission to fail and does not affect the grade. Correctness (respecting every movement, occupancy and capacity rule) is evaluated on its own, independently of the number of turns used. Execution turn count is measured and reported separately, only to assess how well-optimized a solution is.

© 2026 Association 42. - All rights reserved

7.7 Performance Benchmarks

Benchmarks are evaluated only on the maps shipped with this subject. Each map has a single reference target: the **optimal number of turns**, i.e. the fewest turns physically possible given the map's topology and capacity constraints. There is no separate "easy/medium/hard range" — the table below is the only reference.

- **Easy Maps**:
  - Linear path with 2 drones: Optimum = 4 turns
  - Simple fork with 4 drones: Optimum = 6 turns
  - Basic capacity with 4 drones: Optimum = 3 turns
- **Medium Maps**:
  - Dead end trap with 5 drones: Optimum = 8 turns
  - Circular loop with 6 drones: Optimum = 10 turns
  - Priority puzzle with 5 drones: Optimum = 8 turns
- **Hard Maps**:
  - Maze nightmare with 8 drones: Optimum = 13 turns
  - Capacity hell with 12 drones: Optimum = 16 turns
  - Ultimate challenge with 15 drones: Optimum = 26 turns
- **Challenger Map** (optional — for exceptional implementations):
  - The Impossible Dream with 25 drones: **Reference record: 45 turns**
  - This quasi-unsolvable challenge is designed for algorithmic research and optimization.
  - Solving this map demonstrates exceptional pathfinding and optimization skills.
  - **Note**: This level is purely optional and does not affect your grade.



These benchmarks are provided as optimization targets to help you evaluate your algorithm's performance. Meeting a map's optimum demonstrates a well-optimized implementation; missing it is not a failure, but will be recorded during peer evaluation. The evaluation scale checks, independently: that every map runs correctly regardless of its turn count; then awards points if your result stays within 25% of the optimum on every map, and further points if you reach the exact optimum on every map.

© 2026 Association 42. - All rights reserved

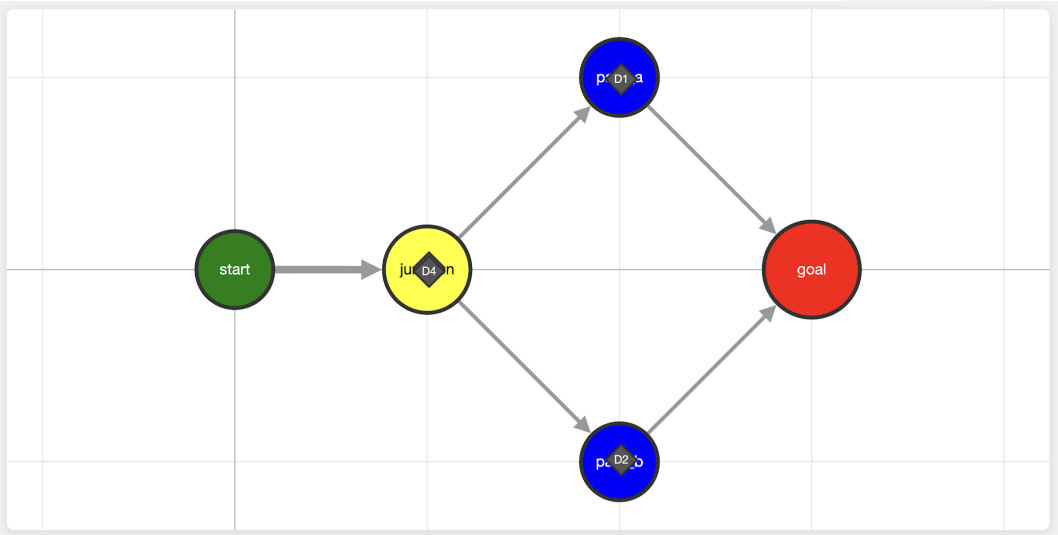


Figure 7.1: Map easy 2

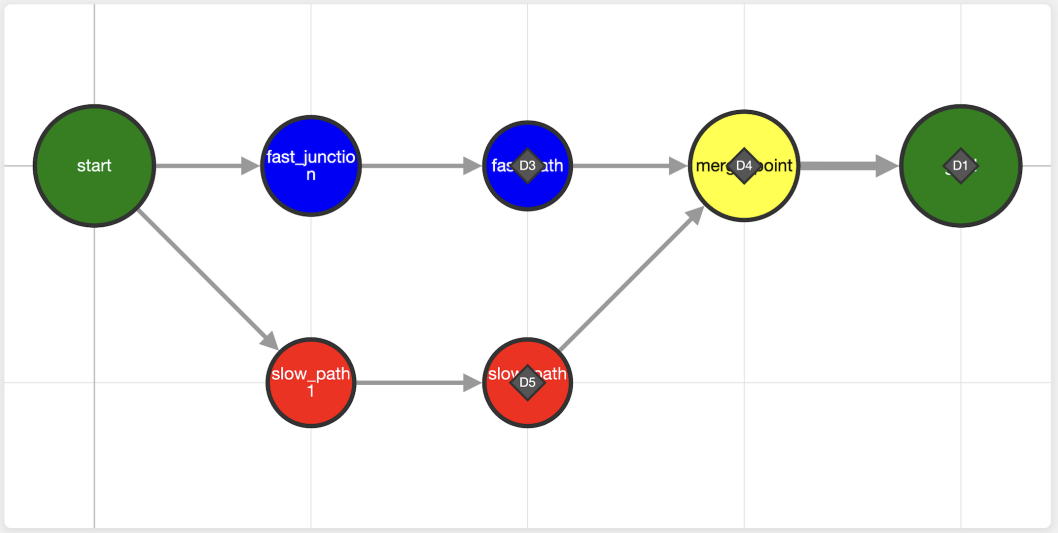


Figure 7.2: Map medium 3

© 2026 Association 42. - All rights reserved

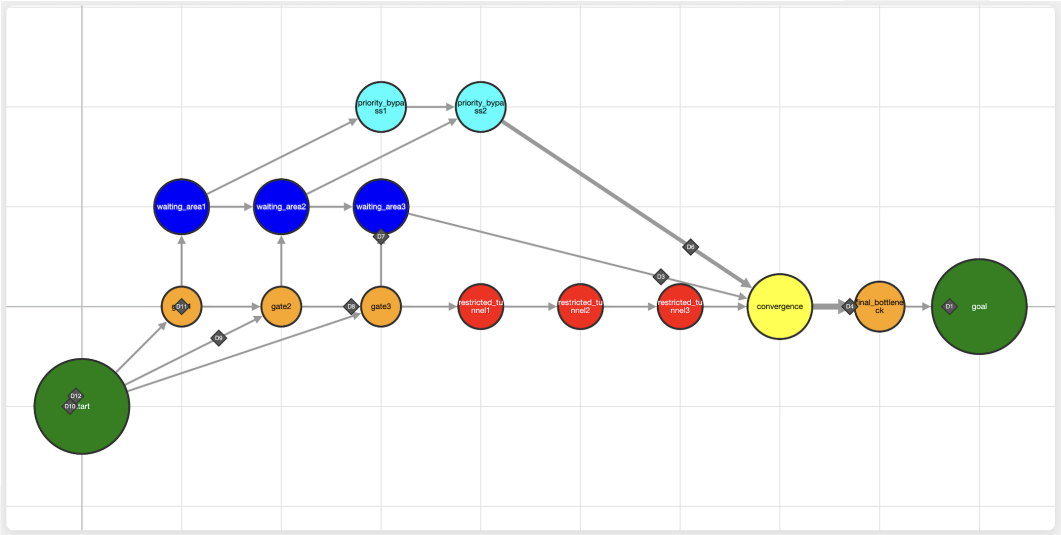


Figure 7.3: Map hard 2

© 2026 Association 42. - All rights reserved

Chapter 8

Readme Requirements

A README.md file must be provided at the root of your Git repository. Its purpose is to allow anyone unfamiliar with the activity (peers, staff, recruiters, etc.) to quickly understand what the activity is about, how to run it, and where to find more information on the topic.
The README.md must include at least:

- The very first line must be italicized and read: *This activity has been created as part of the 42 curriculum by <login1>[, <login2>[, <login3>[...]]].*
- A "**Description**" section that clearly presents the activity, including its goal and a brief overview.
- An "**Instructions**" section containing any relevant information about compilation, installation, and/or execution.
- A "**Resources**" section listing classic references related to the topic (documentation, articles, tutorials, etc.), as well as a description of how AI was used — specifying for which tasks and which parts of the activity.

➠ **Additional sections may be required depending on the activity** (e.g., usage examples, feature list, technical choices, etc.).

Any required additions will be explicitly listed below.

- A detailed description of your algorithm choices and implementation strategy must also be included.
- Documentation of the visual representation features and how they enhance the user experience.
- Example input and expected output demonstrating the program's functionality.



Your README must be written in English.

© 2026 Association 42. - All rights reserved

Chapter 9

Submission and peer-evaluation

Turn in your assignment in your `Git` repository as usual. Only the work inside your repository will be evaluated during the peer-evaluation. Don't hesitate to double check the names of your files to ensure they are correct.



Place all your files at the root of your repository.

A fully working simulation written in Python, including:

- A parser for the input file format.
- A simulation engine respecting movement and zone rules.
- A pathfinding algorithm (or multiple) capable of minimizing total turns.
- A visual representation system (terminal colors and/or graphical interface).
- A terminal or log output that follows the specified format.



Note that we may ask you to explain your code or possibly even to write some code. Make sure to be prepared for this.



Evaluation maps may be different from the ones provided in the subject.

During the review, a brief **modification of the activity** may occasionally be requested. This could involve a minor behavior change, a few lines of code to write or rewrite, or an easy-to-add feature.

While this step may **not be applicable to every activity**, you must be prepared for it if it is mentioned in the review guidelines.

This step is meant to verify your actual understanding of a specific part of the activity. The modification can be performed in any development environment you choose (e.g., your usual

© 2026 Association 42. - All rights reserved

setup), and it should be feasible within a few minutes — unless a specific timeframe is defined as part of the review.
You can, for example, be asked to make a small update to a function or script, modify a display, or adjust a data structure to store new information, etc.

The details (scope, target, etc.) will be specified in the **review guidelines** and may vary from one review to another for the same activity.

© 2026 Association 42. - All rights reserved